Microservices Dependency Analysis Guide

Complete Maven Dependency Reference — What, Why, Where, How

Real-Time Order Processing System

Version: 2.0 | **Date:** March 2026 **Spring Boot:** 3.2.0 | **Spring Cloud:** 2023.0.0 | **Java:** 17

Table of Contents

1. Dependency Overview Matrix
2. Common Events Module
3. Eureka Server
4. API Gateway
5. Auth Service
6. User Service
7. Product Service
8. Order Service
9. Inventory Service
10. Payment Service
11. Notification Service
12. Infrastructure-Level Dependencies
13. Advanced Dependency Insights
14. Architecture-Level Justification

---

1. Dependency Overview Matrix

```
WHICH SERVICE NEEDS WHICH DEPENDENCY?

+————————————————————+———————+———————+———————+———————+———————+———————+———————+———————+
| Dependency         |Eureka|Gate  | Auth | User |Prod. |Order |Inven.|Pay.  |Notif.|
|                    |Server| way  | Svc  | Svc  | Svc  | Svc  | Svc  | Svc  | Svc  |
+————————————————————+———————+———————+———————+———————+———————+———————+———————+———————+———————+
| spring-boot-parent |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| spring-cloud-bom   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| starter-web        |      |      |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| starter-gateway    |      |  ✓   |      |      |      |      |      |      |      |
| starter-data-jpa   |      |      |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| starter-data-redis |      |      |      |      |  ✓   |      |  ✓   |      |      |
| data-redis-reactive|      |  ✓   |      |      |      |      |      |      |      |
| starter-security   |      |      |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |      |  ✓   |
| starter-validation |      |      |  ✓   |  ✓   |  ✓   |  ✓   |      |      |      |
| starter-actuator   |      |  ✓   |      |      |      |      |      |      |      |
| eureka-server      |  ✓   |      |      |      |      |      |      |      |      |
| eureka-client      |      |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| spring-kafka       |      |      |      |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| mysql-connector-j  |      |      |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |  ✓   |
| jjwt (api+impl+jack.)|    |  ✓   |  ✓   |      |      |      |      |      |      |
| resilience4j-reactor|     |  ✓   |      |      |      |      |      |      |      |
```

```
| jackson-databind      |      |      |      |      | ✓   |      |      |      |      |
| jackson-jsr310        |      |      |      |      | ✓   |      |      |      |      |
| common-events         |      |      |      | ✓   | ✓   | ✓   | ✓   | ✓   | ✓   |
+———————————————————+———————+———————+———————+———————+———————+———————+———————+———————+———————+
  Total dependencies:      1     6      7      8     11      7      8      6      7
```

---

2. Common Events Module

pom.xml (No Spring — Pure Java)

```
<groupId>com.orderprocessing</groupId>
<artifactId>common-events</artifactId>
<version>1.0.0</version>
<!-- NO parent spring-boot-starter-parent — this is a plain Java library -->

<dependencies>
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.16.0</version>
    </dependency>
    <dependency>
        <groupId>com.fasterxml.jackson.datatype</groupId>
        <artifactId>jackson-datatype-jsr310</artifactId>
        <version>2.16.0</version>
    </dependency>
</dependencies>
```

Dependency 1: jackson-databind

Aspect	Detail
WHAT	Core Jackson library for JSON serialization/deserialization
WHY	Event objects (OrderEvent, PaymentEvent) must be serialized to JSON when sent to Kafka and deserialized when consumed. Jackson is the JSON processor.
WHERE	Every event class — Jackson annotations (@JsonProperty, @JsonIgnoreProperties) and Kafka's JsonSerializer/JsonDeserializer use this under the hood
HOW	Jackson's ObjectMapper reads Java object fields via reflection → generates JSON string `{"orderId":42,"status":"PENDING"}`. On consumption, it reads JSON → constructs Java object via matching field names.
IF REMOVED	All Kafka serialization fails. Producers can't convert OrderEvent to JSON bytes. Consumers can't reconstruct event objects from Kafka messages.

Dependency 2: jackson-datatype-jsr310

Aspect	Detail

WHAT	Jackson module for Java 8 date/time types (LocalDateTime, Instant, etc.)
WHY	Event classes use `LocalDateTime timestamp` field. Without this module, Jackson cannot serialize/deserialize Java 8 date types and throws `InvalidDefinitionException`.
WHERE	Every event's `timestamp` field — `private LocalDateTime timestamp;`
HOW	Registers custom serializers/deserializers for java.time classes. LocalDateTime becomes `"2026-03-03T14:30:00"` instead of `{"year":2026,"month":3,...}` (the ugly array format).
IF REMOVED	`LocalDateTime` fields serialize as nested arrays `[2026,3,3,14,30,0]` instead of ISO-8601 strings. Consumer deserialization may break with incompatible format errors.

WHY NO SPRING PARENT? Common-events is a plain Java library — no REST endpoints, no Spring context, no auto-configuration. Adding spring-boot-starter-parent would bloat it with 200+ transitive dependencies it doesn't need. It's imported as a compile-time dependency by 6 services.

---

3. Eureka Server

pom.xml

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
</parent>

<properties>
    <spring-cloud.version>2023.0.0</spring-cloud.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

TOTAL: 1 explicit dependency (simplest service in the project)

Dependency 1: spring-cloud-starter-netflix-eureka-server

Aspect	Detail
WHAT	Spring Cloud Netflix Eureka Server — service discovery and registration server
WHY	All microservices register their hostname:port with Eureka. The API Gateway queries Eureka to resolve `lb://order-service` → `http://172.18.0.7:8084`. Without it, every service URL must be hardcoded.
WHERE	`EurekaServerApplication.java` — `@EnableEurekaServer` annotation. `application.yml` — `register-with-eureka: false`, `fetch-registry: false`.
HOW	Internally: embeds a Jersey-based REST API that maintains an in-memory registry. Services send heartbeats (every 30s). If Eureka receives no heartbeat for 90s, it evicts the instance. Gateway calls `GET /eureka/apps/{serviceName}` to discover instances.
IF REMOVED	The module won't compile — `@EnableEurekaServer` doesn't exist. Gateway's `lb://` routes fail with `ServiceNotFoundException`. All services must hardcode peer URLs.

Transitive dependencies pulled in (40+):

```
spring-boot-starter-web (Tomcat, Spring MVC)
spring-boot-starter-jersey (JAX-RS for Eureka's REST API)
netflix-eureka-server (core Eureka logic)
jackson-databind (JSON serialization for registry)
spring-cloud-commons (service registry abstraction)
```

WHY Eureka has NO database? The registry is in-memory only. If Eureka restarts, services re-register within 30 seconds (heartbeat). For HA, deploy 2-3 Eureka instances that replicate with each other — no database needed.

---

4. API Gateway

pom.xml

```
<dependencies>
    <!-- 1 --> <dependency>spring-cloud-starter-gateway</dependency>
    <!-- 2 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 3 --> <dependency>spring-cloud-starter-circuitbreaker-reactor-
resilience4j</dependency>
    <!-- 4 --> <dependency>spring-boot-starter-data-redis-reactive</dependency>
    <!-- 5 --> <dependency>jjwt-api (0.12.3)</dependency>
               <dependency>jjwt-impl (0.12.3, runtime)</dependency>
               <dependency>jjwt-jackson (0.12.3, runtime)</dependency>
    <!-- 6 --> <dependency>spring-boot-starter-actuator</dependency>
</dependencies>
```

TOTAL: 6 dependencies (most unique set in the project)

Dependency 1: spring-cloud-starter-gateway

Aspect	Detail

WHAT	Spring Cloud Gateway — reactive, non-blocking API gateway built on Project Reactor and Netty
WHY	Single entry point for all client requests. Routes HTTP requests to downstream microservices, applies filters (JWT, rate limiting, circuit breaker).
WHERE	`application.yml` — route definitions (predicates, filters). `JwtAuthenticationFilter.java` — extends `AbstractGatewayFilterFactory`. `ApiGatewayApplication.java` — auto-configured.
HOW	Internally uses Netty (not Tomcat) as HTTP server. Incoming request → Route Predicate (match path) → Filter chain (JWT, circuit breaker) → Forwarding via WebClient to downstream service → Response back to client. All non-blocking (reactive).
IF REMOVED	No gateway exists. Frontend must call each microservice directly. No centralized JWT validation, no circuit breakers, no rate limiting, no CORS handling.

CRITICAL: Gateway uses WebFlux (reactive), NOT Spring MVC. You CANNOT add `spring-boot-starter-web` — it conflicts with `spring-cloud-starter-gateway` (both include embedded servers: Tomcat vs Netty).

Dependency 2: spring-cloud-starter-netflix-eureka-client

Aspect	Detail
WHAT	Eureka Client — registers this service with Eureka Server and enables service discovery
WHY	The Gateway uses `lb://order-service` in route URIs. The `lb://` prefix triggers Eureka lookup → resolves to actual instance IP:port. Without it, you'd hardcode `http://localhost:8084`.
WHERE	`application.yml` — `eureka.client.serviceUrl.defaultZone`. Route URIs: `uri: lb://product-service`.
HOW	On startup: registers with Eureka (`POST /eureka/apps/api-gateway`). Every 30s: sends heartbeat. On `lb://` request: calls `DiscoveryClient.getInstances("product-service")` → returns list of IPs → load-balances (round-robin).
IF REMOVED	All `lb://` routes fail. Gateway cannot discover any service. Routes would need `uri: http://product-service:8083` (Docker DNS) — works in Docker but not for scaling (no load balancing).

Dependency 3: spring-cloud-starter-circuitbreaker-reactor-resilience4j

Aspect	Detail
WHAT	Reactor-compatible Resilience4j circuit breaker for Spring Cloud Gateway
WHY	Prevents cascading failures. If Product Service is down, Gateway stops sending requests (opens circuit) and returns a fallback response instead of timing out.
WHERE	`application.yml` — `CircuitBreaker` filter on each route with `fallbackUri: forward:/fallback/generic`. `ResilienceConfig.java` — circuit breaker and rate limiter configuration. `FallbackController.java` — returns friendly error responses.
HOW	Tracks failure rate in a sliding window of N requests. If failure rate > threshold (e.g., 50%): circuit OPENS → all requests immediately fail with fallback. After wait duration: HALF-OPEN → allows test requests. If they succeed: circuit CLOSES.

IF REMOVED	No circuit breakers. If Product Service has a 5-second timeout, ALL gateway requests to products block for 5 seconds → thread pool exhausted → entire gateway unresponsive → cascading failure to ALL services.

WHY `-reactor`? Gateway uses WebFlux (reactive). Standard Resilience4j uses blocking calls. The reactor variant integrates with Mono/Flux reactive types that Gateway uses internally.

Dependency 4: spring-boot-starter-data-redis-reactive

Aspect	Detail
WHAT	Reactive Redis client (Lettuce) for non-blocking Redis operations
WHY	Rate limiting in the Gateway stores counters in Redis. Since Gateway is reactive (Netty), it needs a reactive Redis client — blocking Redis calls would defeat the purpose.
WHERE	Resilience4j rate limiter backed by Redis. `application.yml` — `spring.data.redis.host: redis`.
HOW	Uses Lettuce (not Jedis) which supports non-blocking I/O via Netty. Commands are sent as `Mono<String>` / `Flux<String>`. Rate limiter atomically increments a key per user per time window.
IF REMOVED	Rate limiting falls back to in-memory (per-instance). With 3 Gateway instances, a user could make 300 req/min (100 per instance) instead of 100 total. Redis provides shared state across all Gateway instances.

WHY `-reactive` and not regular `starter-data-redis`? Regular starter includes `spring-boot-starter-web` transitively, which conflicts with Gateway's `spring-cloud-starter-gateway` (Tomcat vs Netty). The reactive variant uses Lettuce's async API compatible with WebFlux.

Dependency 5: jjwt-api + jjwt-impl + jjwt-jackson

Aspect	Detail
WHAT	JJWT (Java JWT) library — JWT creation, parsing, and validation. Split into API (interfaces), impl (logic), jackson (JSON processing).
WHY	`JwtAuthenticationFilter` must validate JWT tokens on every request. It verifies the HMAC-SHA256 signature using a shared secret key, checks token expiry, and extracts username + role claims.
WHERE	`JwtUtil.java` — `isTokenValid()`, `extractUsername()`, `extractRole()`. `JwtAuthenticationFilter.java` — calls JwtUtil for every non-open endpoint.
HOW	`jjwt-api`: Defines builder interfaces (Jwts.parser()). `jjwt-impl`: HMAC-SHA256 signature generation/verification (`runtime` scope — not exposed to compile-time API). `jjwt-jackson`: Uses Jackson to serialize/deserialize JWT payload JSON.
IF REMOVED	Gateway cannot validate JWT tokens. All authenticated endpoints become either open (security breach) or universally blocked.

WHY jjwt-impl is `runtime` scope? You code against `jjwt-api` interfaces (compile-time). The implementation is discovered at runtime via Java SPI. This follows the API/Implementation separation principle — you can swap JWT libraries without changing application code.

Dependency 6: spring-boot-starter-actuator

Aspect	Detail
WHAT	Production-ready features: health checks, metrics, info endpoints
WHY	Docker healthcheck uses `/actuator/health` to verify Gateway is running. Prometheus can scrape `/actuator/prometheus` for monitoring.
WHERE	Docker Compose — `healthcheck: curl -f http://localhost:8080/actuator/health`. Resilience4j metrics exposed via actuator.
HOW	Auto-configures health indicators for each connected system (Redis, Eureka). Exposes `/actuator/health` (UP/DOWN), `/actuator/metrics` (JVM, HTTP, custom), `/actuator/info`.
IF REMOVED	No health checks. Docker can't detect if Gateway is truly ready. No metrics for monitoring dashboards.

---

5. Auth Service

pom.xml

```
<dependencies>
    <!-- 1 --> <dependency>spring-boot-starter-web</dependency>
    <!-- 2 --> <dependency>spring-boot-starter-security</dependency>
    <!-- 3 --> <dependency>spring-boot-starter-data-jpa</dependency>
    <!-- 4 --> <dependency>spring-boot-starter-validation</dependency>
    <!-- 5 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 6 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 7 --> <dependency>jjwt-api + jjwt-impl + jjwt-jackson (0.12.3)</dependency>
</dependencies>
```

Dependency 1: spring-boot-starter-web

Aspect	Detail
WHAT	Embedded Tomcat + Spring MVC + Jackson JSON — everything needed for REST APIs
WHY	Auth Service exposes REST endpoints: POST /api/auth/signup, POST /api/auth/login, GET /api/auth/validate. Needs an HTTP server and request handling framework.
WHERE	`AuthController.java` — `@RestController`, `@GetMapping`, `@PostMapping`, `@RequestBody`, `ResponseEntity`.
HOW	Embeds Apache Tomcat (default port 8080). Spring MVC's DispatcherServlet routes requests to controllers by matching @RequestMapping paths. Jackson auto-serializes return objects to JSON.
IF REMOVED	No HTTP server starts. No REST endpoints. Auth Service becomes a background application with no way to receive requests.

Transitive dependencies (30+): Spring MVC, Tomcat, Jackson, SLF4J/Logback, SnakeYAML, spring-core, spring-context...

Dependency 2: spring-boot-starter-security

Aspect	Detail
WHAT	Spring Security framework — authentication, authorization, CSRF, session management
WHY	Provides `PasswordEncoder` (BCryptPasswordEncoder) for hashing passwords. Provides `SecurityFilterChain` for endpoint-level access control.
WHERE	`SecurityConfig.java` — `@Bean PasswordEncoder`, `SecurityFilterChain`. `AuthService.java` — `passwordEncoder.encode()` on signup, `passwordEncoder.matches()` on login.
HOW	On startup: Spring creates a filter chain that intercepts every HTTP request. `SecurityConfig` configures: CSRF disabled (stateless API), session STATELESS, /api/auth/** permitAll. BCryptPasswordEncoder uses Blowfish cipher with 12 rounds + random salt.
IF REMOVED	No PasswordEncoder bean → BCrypt hashing unavailable → passwords stored in plaintext (critical security vulnerability). Also, Spring Boot auto-generates a random password and blocks all endpoints by default.

Dependency 3: spring-boot-starter-data-jpa

Aspect	Detail
WHAT	JPA (Java Persistence API) + Hibernate ORM + Spring Data repositories
WHY	Auth Service stores user credentials in MySQL. JPA provides ORM mapping (`@Entity`), and Spring Data provides `JpaRepository<UserCredential, Long>` which auto-generates SQL queries.
WHERE	`UserCredential.java` — `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`. `UserCredentialRepository.java` — `extends JpaRepository`, custom methods: `findByUsername()`, `existsByEmail()`.
HOW	Hibernate maps Java objects to SQL tables. `repository.save(user)` → `INSERT INTO users (username, email, password, role) VALUES (...)`. Spring Data generates implementations at runtime using JDK dynamic proxies based on method names.
IF REMOVED	No ORM. Must write raw JDBC queries: `PreparedStatement ps = conn.prepareStatement("INSERT INTO...")`. No entity mapping, no transaction management, no repository abstraction.

Dependency 4: spring-boot-starter-validation

Aspect	Detail
WHAT	Jakarta Bean Validation (Hibernate Validator) — `@NotNull`, `@Size`, `@Email`, `@Valid`
WHY	Validates incoming DTOs before processing. `@Valid @RequestBody SignupRequest request` ensures username/email/password meet requirements before hitting the service layer.
WHERE	`SignupRequest.java` — `@NotBlank`, `@Email`, `@Size(min=6)`. `AuthController.java` — `@Valid` on method parameters.
HOW	Spring MVC's `MethodArgumentNotValidException` handler invokes Hibernate Validator. If any constraint fails → 400 Bad Request with field-level error messages. Validation happens BEFORE the controller method body executes.

IF REMOVED	No input validation. Empty username, invalid email, 1-character password all pass through. SQL injection risk increases. Must validate manually in every service method.

Dependency 5: mysql-connector-j

Aspect	Detail
WHAT	MySQL JDBC driver — enables Java to communicate with MySQL over TCP
WHY	JPA/Hibernate requires a JDBC driver to execute SQL. `mysql-connector-j` translates Hibernate's SQL into MySQL wire protocol commands.
WHERE	`application.yml` — `spring.datasource.url: jdbc:mysql://...`. Used implicitly by every JPA repository operation.
HOW	Registers as a JDBC driver via Java SPI. HikariCP connection pool creates TCP connections to MySQL:3306. Hibernate generates SQL → sends via JDBC driver → MySQL returns result sets → driver maps to Java types.
IF REMOVED	`Cannot load driver class: com.mysql.cj.jdbc.Driver`. Spring Boot fails to create the DataSource → all JPA operations fail → application won't start.

WHY `runtime` scope? Application code never directly references MySQL classes. JPA/Hibernate abstracts the database. The driver is loaded dynamically at runtime via `Class.forName()`.

Dependency 6: eureka-client

Same as Gateway's eureka-client (see Section 4). Registers Auth Service with Eureka so Gateway can discover it via `lb://auth-service`.

Dependency 7: jjwt (api + impl + jackson)

Same as Gateway (see Section 4). But in Auth Service, JJWT is used for **generation** (not just validation): `Jwts.builder().subject(username).claim("role", role).signWith(key).compact()`. Auth Service is the ONLY service that generates tokens.

---

6. User Service

pom.xml

```
<dependencies>
    <!-- 1 --> <dependency>spring-boot-starter-web</dependency>
    <!-- 2 --> <dependency>spring-boot-starter-data-jpa</dependency>
    <!-- 3 --> <dependency>spring-boot-starter-validation</dependency>
    <!-- 4 --> <dependency>spring-kafka</dependency>
    <!-- 5 --> <dependency>common-events (1.0.0)</dependency>
    <!-- 6 --> <dependency>spring-boot-starter-security</dependency>
    <!-- 7 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 8 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
</dependencies>
```

Dependency 4: spring-kafka

Aspect	Detail
WHAT	Spring for Apache Kafka — simplified Kafka producer/consumer integration
WHY	User Service publishes user-events (profile created, updated) and audit-logs to Kafka for downstream consumption by Notification Service.
WHERE	`UserEventProducer.java` — `KafkaTemplate<String, UserEvent>.send("user-events", event)`. `application.yml` — `spring.kafka.bootstrap-servers`, producer serializer config.
HOW	Wraps Apache Kafka's `KafkaProducer` with Spring's `KafkaTemplate`. Auto-configures serializers, partitioners, and error handlers. `send()` is async — returns `CompletableFuture<SendResult>`.
IF REMOVED	User Service cannot publish events. Notification Service never knows about user profile changes. Audit trail for user actions is lost.

Dependency 5: common-events (1.0.0)

Aspect	Detail
WHAT	Internal Maven module (com.orderprocessing:common-events) — shared Kafka event POJOs
WHY	`UserEventProducer` sends `UserEvent` and `AuditLogEvent` objects. These classes are defined in common-events, ensuring type consistency between producer (User Service) and consumer (Notification Service).
WHERE	`UserEventProducer.java` — `import com.orderprocessing.commonevents.event.UserEvent`.
HOW	Installed to local Maven repository (`mvn install`). Each consumer service depends on the same artifact, guaranteeing identical class structure for Kafka's JSON serializer/deserializer.
IF REMOVED	`ClassNotFoundException: UserEvent`. Compilation fails. Even if you copy the class locally, version drift between services causes deserialization failures.

---

7. Product Service

pom.xml

```
<dependencies>
    <!-- 1-3 --> <dependency>starter-web, starter-data-jpa, starter-validation</dependency>
    <!-- 4 --> <dependency>spring-boot-starter-data-redis</dependency>
    <!-- 5 --> <dependency>spring-kafka</dependency>
    <!-- 6 --> <dependency>common-events (1.0.0)</dependency>
    <!-- 7 --> <dependency>spring-boot-starter-security</dependency>
    <!-- 8 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 9 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 10 --> <dependency>jackson-databind</dependency>
    <!-- 11 --> <dependency>jackson-datatype-jsr310</dependency>
</dependencies>
```

TOTAL: 11 dependencies — most dependencies in any single service

Dependency 4: spring-boot-starter-data-redis

Aspect	Detail
WHAT	Spring Data Redis — Redis client (Lettuce), RedisTemplate, and Spring Cache integration
WHY	Product Service implements Cache-Aside pattern: `@Cacheable` stores product data in Redis. `RedisCacheManager` manages cache lifecycle. `GenericJackson2JsonRedisSerializer` stores data as human-readable JSON.
WHERE	`RedisConfig.java` — `CacheManager` bean with per-cache TTL. `ProductService.java` — `@Cacheable`, `@CachePut`, `@CacheEvict`. `application.yml` — `spring.cache.type: redis`.
HOW	Spring Cache abstraction intercepts annotated methods via AOP proxy. On `@Cacheable`: generates cache key → calls `redisTemplate.opsForValue().get(key)` → if nil → invokes method → `set(key, result, ttl)`. Uses Lettuce client which is thread-safe (single connection for all threads).
IF REMOVED	All @Cacheable annotations are ignored (no CacheManager). Every `getProductById()` call hits MySQL. Response time increases from ~0.5ms (cache hit) to ~5ms (DB query). Under heavy load: MySQL becomes the bottleneck.

WHY `starter-data-redis` (not reactive)? Product Service uses Spring MVC (blocking). The regular Redis starter uses Lettuce's synchronous API. Gateway needs the reactive variant because it runs on Netty.

Dependencies 10-11: jackson-databind + jackson-datatype-jsr310

Aspect	Detail
WHAT	Jackson JSON processor + Java 8 date/time module
WHY	`GenericJackson2JsonRedisSerializer` in `RedisConfig` uses Jackson to serialize product data as JSON in Redis. The JSR-310 module handles `LocalDateTime` fields in entities.
WHERE	`RedisConfig.java` — `GenericJackson2JsonRedisSerializer()`. Kafka `JsonSerializer` also uses Jackson.
HOW	When `@Cacheable` stores a ProductDto in Redis: Jackson serializes it to `{"@class":"...ProductDto","id":1,"name":"Laptop","price":999.99}`. The `@class` property enables polymorphic deserialization.
IF REMOVED	Jackson is already a transitive dependency of `starter-web`, so direct declaration is technically optional. But explicit declaration ensures version consistency and makes the dependency visible in the POM.

---

8. Order Service

pom.xml

```
<dependencies>
    <!-- 1-3 --> <dependency>starter-web, starter-data-jpa, starter-validation</dependency>
    <!-- 4 --> <dependency>spring-kafka</dependency>
    <!-- 5 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 6 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 7 --> <dependency>common-events (1.0.0)</dependency>
```

```
    <!-- 8 --> <dependency>spring-boot-starter-security</dependency>
</dependencies>
```

WHY NO REDIS? Order Service doesn't cache data or use distributed locks. Orders are always read fresh from the database (consistency is critical — you never want stale order status).

WHY KAFKA IS CRITICAL HERE? Order Service is the Saga INITIATOR. It publishes `order-created` → triggers the entire inventory → payment → notification chain. It also consumes `payment-success/failure` to update order status.

---

9. Inventory Service

pom.xml

```
<dependencies>
    <!-- 1 --> <dependency>spring-boot-starter-web</dependency>
    <!-- 2 --> <dependency>spring-boot-starter-data-jpa</dependency>
    <!-- 3 --> <dependency>spring-boot-starter-data-redis</dependency>
    <!-- 4 --> <dependency>spring-boot-starter-security</dependency>
    <!-- 5 --> <dependency>spring-kafka</dependency>
    <!-- 6 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 7 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 8 --> <dependency>common-events (1.0.0)</dependency>
</dependencies>
```

WHY REDIS + KAFKA + JPA (all three)?

```
Inventory Service is the most complex service because it:
  1. Consumes Kafka events (order-created, order-cancelled, product-created)
  2. Uses Redis SETNX for distributed locking (prevents overselling)
  3. Uses JPA @Version for optimistic locking (second safety layer)
  4. Produces Kafka events (inventory-updated)

WITHOUT KAFKA: Can't receive order events → orders never processed
WITHOUT REDIS: No distributed lock → race condition → overselling
WITHOUT JPA: Can't persist stock levels → data lost on restart
```

Dependency 3: spring-boot-starter-data-redis — In Inventory Service, Redis is used DIFFERENTLY than in Product Service:

- Product Service: `RedisCacheManager` (Spring Cache abstraction — `@Cacheable`)
- Inventory Service: `StringRedisTemplate` (raw Redis commands — `setIfAbsent()` , `get()` , `delete()`)

```
// Product Service (caching):
@Cacheable(value = "product", key = "#id")
public ProductDto getProductById(Long id) { ... }
```

```
// Inventory Service (locking):
redisTemplate.opsForValue().setIfAbsent(lockKey, lockValue, Duration.ofSeconds(30));
```

Same dependency, entirely different usage pattern. This is why the dependency exists in both services.

---

10. Payment Service

pom.xml

```
<dependencies>
    <!-- 1 --> <dependency>spring-boot-starter-web</dependency>
    <!-- 2 --> <dependency>spring-boot-starter-data-jpa</dependency>
    <!-- 3 --> <dependency>spring-kafka</dependency>
    <!-- 4 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 5 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 6 --> <dependency>common-events (1.0.0)</dependency>
</dependencies>
```

TOTAL: 6 dependencies — leanest domain service

WHY NO SPRING SECURITY? Payment Service doesn't expose admin endpoints and doesn't enforce role-based access. All inbound requests come through the Gateway (already JWT-validated). Kafka events are internal (no authentication needed). Adding Security would add unnecessary complexity.

WHY NO REDIS? Payment processing is write-heavy (save payment record) and needs strict consistency. Caching payment data could cause a user to see "payment pending" when it's actually completed. Distributed locks aren't needed because payment events are processed sequentially per consumer partition.

WHY NO VALIDATION? Payment Service doesn't receive direct user input via REST. It consumes Kafka events (already validated by Order Service). The `InventoryEvent` consumed is generated by trusted internal services.

---

11. Notification Service

pom.xml

```
<dependencies>
    <!-- 1 --> <dependency>spring-boot-starter-web</dependency>
    <!-- 2 --> <dependency>spring-boot-starter-data-jpa</dependency>
    <!-- 3 --> <dependency>spring-kafka</dependency>
    <!-- 4 --> <dependency>mysql-connector-j (runtime)</dependency>
    <!-- 5 --> <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- 6 --> <dependency>common-events (1.0.0)</dependency>
    <!-- 7 --> <dependency>spring-boot-starter-security</dependency>
</dependencies>
```

WHY JPA in a Notification Service? This service persists notifications to a database (notification_db) for history/audit. Users can query `GET /api/notifications` to see past notifications. If notifications were fire-and-forget (email only), JPA would be unnecessary.

WHY SECURITY? Notification Service has admin endpoints: `GET /admin/audit-logs` restricted to ADMIN role via `@PreAuthorize("hasRole('ADMIN')")`.

---

12. Infrastructure-Level Dependencies

12.1 spring-boot-starter-parent (3.2.0)

```
WHAT: Parent POM that provides:
  - Default Java version (17)
  - Dependency version management (declares versions for 400+ libraries)
  - Plugin management (maven-compiler-plugin, maven-surefire-plugin)
  - Default configuration (UTF-8 encoding, resource filtering)

WHY: Without it, you declare versions for EVERY dependency manually.
  WITH parent:  <artifactId>spring-boot-starter-web</artifactId>  (no version!)
  WITHOUT:      <artifactId>spring-boot-starter-web</artifactId>
                <version>3.2.0</version>

WHICH SERVICES: ALL 9 deployable services (not common-events)

PERFORMANCE: Zero runtime impact — parent POM is a build-time artifact only.
```

12.2 spring-cloud-dependencies BOM (2023.0.0)

```
WHAT: Bill of Materials (BOM) that declares compatible versions for ALL
      Spring Cloud libraries (Eureka, Gateway, Circuit Breaker, etc.)

WHY: Spring Cloud 2023.0.0 is tested against Spring Boot 3.2.x.
  If you mix Spring Cloud 2022.0.0 with Spring Boot 3.2.0 → version conflicts
  → runtime ClassNotFoundException → mysterious failures.

WHICH SERVICES: ALL 9 deployable services (even those without Spring Cloud deps,
  as a safety net for future additions).

HOW: <dependencyManagement> imports the BOM → all spring-cloud-* dependencies
  automatically get compatible versions. No version declared in <dependencies>.
```

12.3 Dependency Distribution Analysis

```
spring-boot-starter-web:           7 services (not Eureka or Gateway)
spring-boot-starter-data-jpa:      7 services (not Eureka or Gateway)
mysql-connector-j:                 7 services (every JPA service)
eureka-client:                     8 services (not Eureka Server itself)
spring-kafka:                      6 services
(User,Product,Order,Inventory,Payment,Notification)
common-events:                     6 services (same as Kafka, minus Eureka and Gateway)
spring-boot-starter-security:      6 services (Auth,User,Product,Order,Inventory,Notification)
spring-boot-starter-validation:    4 services (Auth,User,Product,Order)
spring-boot-starter-data-redis:    2 services (Product, Inventory)
data-redis-reactive:               1 service (Gateway only)
```

```
jjwt:                              2 services (Auth, Gateway)
resilience4j:                      1 service (Gateway only)
actuator:                          1 service (Gateway only)
eureka-server:                     1 service (Eureka only)
```

---

13. Advanced Dependency Insights

13.1 Starter Dependencies vs Direct Dependencies

```
STARTER (spring-boot-starter-web):
  - An empty JAR with ONLY a pom.xml listing curated transitive dependencies
  - Pulls in: spring-webmvc, tomcat, jackson-databind, slf4j-api, logback
  - Benefit: One line in your POM → 30+ compatible libraries
  - You never reference the starter in code — it's purely a build convenience

DIRECT (jackson-databind in common-events):
  - You explicitly declare a specific library with a specific version
  - Used when: no starter exists, or you need a non-Spring library
  - Example: jjwt-api (no Spring starter for JJWT exists)
```

13.2 Transitive Dependencies

```
YOUR POM DECLARES:
  spring-boot-starter-web

MAVEN ACTUALLY DOWNLOADS (60+ JARs):
  spring-boot-starter-web
    ├── spring-webmvc
    │   ├── spring-core
    │   ├── spring-beans
    │   ├── spring-context
    │   └── spring-expression
    ├── spring-boot-starter-tomcat
    │   ├── tomcat-embed-core
    │   ├── tomcat-embed-el
    │   └── tomcat-embed-websocket
    ├── spring-boot-starter-json
    │   ├── jackson-databind
    │   ├── jackson-datatype-jdk8
    │   └── jackson-datatype-jsr310
    └── spring-boot-starter-logging
        ├── logback-classic
        ├── slf4j-api
        └── log4j-to-slf4j

IMPLICATION: Product Service declares jackson-databind explicitly, but it's
already a transitive dep of starter-web. The explicit declaration is
for DOCUMENTATION purposes (makes the dependency visible in the POM).
```

13.3 Dependency Conflict Resolution

```
SCENARIO: common-events uses jackson-databind 2.16.0
          spring-boot-starter-web pulls jackson-databind 2.15.3 (via parent BOM)

MAVEN RESOLUTION: "Nearest wins" — the version declared closest to your POM wins.
  Since spring-boot-starter-parent manages 2.15.3 and common-events explicitly
  declares 2.16.0, the effective version depends on dependency tree order.

SPRING BOOT FIX: <dependencyManagement> in parent POM declares the winning version.
  All transitive Jacksons resolve to the parent-managed version.

BEST PRACTICE: Never declare explicit versions for libraries managed by
  spring-boot-starter-parent. Let the parent manage compatible versions.
  ONLY declare versions for non-Spring libraries (e.g., jjwt 0.12.3).
```

13.4 Version Management Strategy

```
THIS PROJECT'S STRATEGY:

  Layer 1: spring-boot-starter-parent (3.2.0)
    → Manages 400+ library versions (Spring, Jackson, Hibernate, Tomcat, etc.)
    → Single version to upgrade: change parent → all dependencies update

  Layer 2: spring-cloud-dependencies BOM (2023.0.0)
    → Manages all Spring Cloud libraries (Eureka, Gateway, Resilience4j)
    → Must match Spring Boot version (2023.0.0 ↔ Boot 3.2.x)

  Layer 3: Explicit versioning (only for non-managed dependencies)
    → jjwt: 0.12.3 (not managed by Spring Boot parent)
    → jackson in common-events: 2.16.0 (common-events has no parent)

UPGRADE PATH:
  1. Bump spring-boot-starter-parent to 3.3.0
  2. Bump spring-cloud-dependencies to corresponding version (2023.0.x)
  3. Run mvn dependency:tree → check for conflicts
  4. Run tests → verify compatibility
```

13.5 Why Dependency Scope Matters

```
SCOPE: compile (default)
  Available: compile + test + runtime
  Example: spring-boot-starter-web
  WHEN: You import classes from this library in your code

SCOPE: runtime
  Available: test + runtime (NOT compile)
  Example: mysql-connector-j, jjwt-impl
  WHEN: Library is loaded dynamically (JDBC SPI, Java SPI)
  BENEFIT: Compiler error if you accidentally import MySQL-specific code
```

```
          → enforces abstraction (code against JPA, not MySQL)

SCOPE: test
  Available: test only
  Example: spring-boot-starter-test, h2 (in-memory DB for tests)
  WHEN: Only needed for unit/integration tests

SCOPE: provided
  Available: compile (NOT packaged in JAR)
  Example: lombok (annotation processing at compile-time)
  WHEN: Container provides it, or it's compile-time-only tool

SCOPE: import (BOM only)
  Available: dependencyManagement section only
  Example: spring-cloud-dependencies
  WHEN: Importing a BOM to manage versions
```

13.6 Security Risks of Unmanaged Dependencies

```
RISK 1: Known vulnerabilities (CVEs)
  Old Jackson version → CVE-2022-42003 (Remote Code Execution via polymorphic deserialization)
  MITIGATION: Use spring-boot-starter-parent → Spring team patches CVEs promptly

RISK 2: Dependency confusion attacks
  Attacker publishes a malicious package with YOUR internal artifact name to Maven Central
  MITIGATION: Use groupId that matches your domain (com.orderprocessing)

RISK 3: Transitive dependency vulnerabilities
  Your POM is clean, but a transitive dep of spring-kafka has a CVE
  MITIGATION: Run mvn dependency:tree | grep vulnerable-lib
  TOOL: OWASP dependency-check-maven plugin

BEST PRACTICE:
  mvn versions:display-dependency-updates    # Show outdated deps
  mvn dependency:analyze                     # Find unused/undeclared deps
```

13.7 How to Optimize Dependency Footprint

```
1. REMOVE UNUSED DEPENDENCIES:
   mvn dependency:analyze → lists "declared but unused" dependencies
   Example: If Payment Service adds starter-validation but never uses @Valid

2. USE APPROPRIATE SCOPE:
   mysql-connector-j: runtime (not compile)
   jjwt-impl: runtime (not compile)
   This prevents accidental coupling to implementation details

3. EXCLUDE UNWANTED TRANSITIVE DEPENDENCIES:
   <dependency>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-web</artifactId>
```

```
        <exclusions>
            <exclusion>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-tomcat</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <!-- Use Jetty instead of Tomcat -->
    <dependency>spring-boot-starter-jetty</dependency>

4. MINIMIZE COMMON-EVENTS:
   Only Jackson — no Spring, no Lombok, no unnecessary libraries.
   Every dependency in common-events is inherited by 6 services.
```

---

14. Architecture-Level Justification

14.1 Why Auth Service Needs Security + JWT

```
Auth Service is the IDENTITY PROVIDER for the entire system:
  1. spring-boot-starter-security: BCryptPasswordEncoder for password hashing
     → Without it: passwords stored as plaintext → one breach exposes all accounts
  2. jjwt: JWT token generation (Jwts.builder().signWith(key).compact())
     → Without it: no standard token format → custom token = security risk
  3. Both Auth and Gateway share the same HMAC-SHA256 secret key
     → Auth GENERATES tokens, Gateway VALIDATES them
     → Same jjwt dependency, different usage
```

14.2 Why Order Service Needs Kafka (Not REST)

```
IF ORDER SERVICE USED REST CALLS:
  Order → REST → Inventory → REST → Payment → REST → Notification
  Problem 1: If Payment is down → Order waits forever → user timeout
  Problem 2: Partial completion: Inventory reserved but Payment failed
             → How to release stock? Complex compensation logic.
  Problem 3: Order Service must know URL of Inventory, Payment, Notification
             → Changes in one service require changes in Order Service

WITH KAFKA:
  Order → publishes event → returns 200 immediately
  Each downstream service processes independently
  Kafka retains events until consumed (fault tolerance)
  New services can subscribe without modifying Order Service
```

14.3 Why Product + Inventory Need Redis (Different Reasons)

```
PRODUCT SERVICE: CACHING
  Problem: 90% of requests are product reads (catalog browsing)
  Solution: Redis cache with @Cacheable → 90% of reads skip MySQL
  Dependency: spring-boot-starter-data-redis
```

```
  Usage: RedisCacheManager, GenericJackson2JsonRedisSerializer

INVENTORY SERVICE: DISTRIBUTED LOCKING
  Problem: 100 concurrent reservations for same product → overselling
  Solution: Redis SETNX lock → only one thread enters critical section
  Dependency: spring-boot-starter-data-redis (same jar, different usage)
  Usage: StringRedisTemplate, setIfAbsent(), get(), delete()

SAME DEPENDENCY, COMPLETELY DIFFERENT ARCHITECTURAL PURPOSE.
```

14.4 Why Gateway Needs WebFlux (Not Spring MVC)

```
GATEWAY IS A PROXY — it forwards requests, NOT a business service.

SPRING MVC (BLOCKING):
  Thread-per-request model
  100 concurrent requests → 100 threads → if downstream is slow (5s)
  → 100 threads blocked → 101st request: "no threads available" → 503

WEBFLUX (NON-BLOCKING):
  Event loop model (Netty, like Node.js)
  100 concurrent requests → 1-4 threads
  When downstream is slow → thread released back to pool → handles other requests
  → Can handle 10,000+ concurrent connections on 4 threads

CONCLUSION: Gateway MUST be non-blocking because it proxies ALL traffic.
  If Gateway blocks → entire system blocks.
```

14.5 Why Eureka Server Does NOT Need a Database

```
REGISTRY IS EPHEMERAL:
  Services re-register within 30 seconds via heartbeats.
  If Eureka restarts → memory cleared → services re-register.
  No data to persist → no database needed.

FOR HIGH AVAILABILITY:
  Deploy 2-3 Eureka instances that replicate with each other (peer awareness).
  NOT database replication — Eureka-to-Eureka HTTP replication.
```

14.6 Why Payment Service Has the Fewest Dependencies

```
Payment Service is intentionally lean:
  1. No Security: All requests come through authenticated Gateway
  2. No Validation: Consumes trusted Kafka events (validated upstream)
  3. No Redis: Payment records need strict consistency (no caching)
  4. No Admin endpoints: No @PreAuthorize needed

This follows the Minimal Dependencies principle:
  Every dependency is an attack surface, maintenance burden, and startup cost.
  Only include what you ACTUALLY use.
```

End of Document

This guide provides architectural reasoning for every Maven dependency in the project. Use the Dependency Matrix (Section 1) as a quick reference, the per-service sections (2-11) for deep understanding, and the Advanced Insights (Section 13) for interview preparation.